

School of Electronics Engineering

Vellore Institute of Technology, Chennai

Vandalur – Kelambakkam Road, Chennai – 600127

October 2026

REAL-TIME MULTIMODAL AI COMPANION ('JARVIS')

Submitted by

AVYUKTH M A (24BEC1228)

G HARISHANKARVEL (24BLC1243)

DHIVYESH S (24BEC1235)

A project report submitted to

Dr. Vijayarajan R

SCHOOL OF ELECTRONICS ENGINEERING

in partial fulfilment of the requirements for the course of

BECE309L – Artificial Intelligence and Machine Learning

Acknowledgement

We would like to share our regards with our project guide, Dr. Vijayarajan R, Professor in the School of Electronics Engineering (SENSE) at Vellore Institute of Technology, Chennai. His hands-on advice, explanations and friendly guidance kept us going through every stage of building this project.

Signatures:

Avyukth (24BEC1228)

Harishankarvel (24BLC1243)

Dhivyesh (24BEC1235)

Abstract

Most everyday voice helpers or chat apps are just trapped inside a basic text entry box or tied to a single smart speaker. They lack vision and cannot respond if you wave your hands or make a physical sign or gesture. To solve this, our team built “JARVIS”. It operates in a standard web browser without need of any expensive hardware or software.

Frontend uses Three.js alongside React Three Fiber. Text answers stream back word by word from Groq (llama-3.3-70b-versatile) or Google Gemini. To give the software sight, we feed standard webcam video into MediaPipe and YOLOv8. This vision pipeline follows 21 individual hand joints and detects 5 distinct gestures at a steady rate of 10 frames per second. As soon as a gesture is caught on camera, that data injects directly into the active prompt pipeline.

On the audio side, a custom background queue splits model outputs into proper sentence chunks using standard punctuation checks. Edge-TTS generates spoken audio for every single word, streaming Base64-encoded MP3 packages back to the browser for ordered playback alongside synchronized subtitle highlights. If a user lifts an open palm toward the screen, JARVIS stops talking immediately, killing text generation and wiping out audio buffers within the next 42 milliseconds.

Codes, Standards & SDG Mapping

Standards & Protocols

RFC 6455 (WebSocket Protocol): It provides an open, continuous two-way communication channel which is used to stream live video frames, gestures, and audio.

W3C WebRTC: It allows web browsers to receive raw webcam feeds and microphone inputs without relying on third-party softwares .

ISO/IEC 22989:2022: It provides standardized technical definitions for machine learning algorithms, model behaviours, and inference execution pipelines.

Mapped SDG Table

Mapped SDG	Why It Matters	Real-World Impact
SDG 3: Good Health & Wellbeing	A visual companion helps in reducing feelings of isolation and actively engages the user's mind.	Elders and individuals who feel isolated actively engage in conversations with a person-like software
SDG 9: Industry & Innovation	It proves complex 3D visual environments and streamed neural outputs are efficient inside ordinary web browsers without need of specialized graphics cards.	Browser based interfaces which do not need any expensive hardware can be widely scaled out for everyday users
SDG 10: Reduced Inequalities	Voice and hand gestures along with keyboard input helps the user in choosing input according their convenience	Ease of accessibility is improved for people with physical disabilities.

Table of Contents & Acronyms

Table of Contents

1. Introduction & Project Goals
2. Literature Survey & Research Gaps
3. Methodology & System Design
4. Implementation Details & Modules
5. Testing Results & Performance Analysis
6. Conclusion and Source Code
7. References

List of Acronyms

Acronym	What It Stands For
LLM	Large Language Model
LPU	Language Processing Unit
YOLO	You Only Look Once (Object Detection Architecture)
TTS	Text-to-Speech
FPS	Frames Per Second
WebRTC	Web Real-Time Communication
WebGL	Web Graphics Library
LERP	Linear Interpolation

CHAPTER 1: Introduction & Project Goals

1.1 Background and Motivation

Current LLMs are exceptionally smart yet the way of interaction with them has not updated much. Most users are still stuck typing into standard text boxes or talking to small speakers sitting on their desks. These setups remain completely blind to user's surroundings.

JARVIS bridges this gap. It operates as an interactive digital companion that can see and speak with you directly inside a standard web browser. By combining live webcam hand tracking with phonetic text formatting, the entire project runs smoothly without requiring extra hardware.

1.2 Main Challenges We Solved

1. **Visual and Spatial Blindness:** Normal chatbots lack camera access, leaving them totally unaware of user surroundings.
2. **Decoupled Vision and Neural Processing:** Conventional computer vision (CV) and large language models(LLMs) act as standalone units making the core AI engine incapable of receiving continuous input of video feed by the web camera
3. **Rigid Monolithic Speech Synthesis:** The entire audio file of the response generated by voice tools is played as a rigid playback along with the response due to which it cannot be interrupted.
4. **Heavy Rendering Overhead:** Detailed 3D avatars typically demand heavy hardware along with plugins that tank overall system performance and are complex to use.

1.3 Core Objectives

1. **Client-Side 3D WebGL Rendering:** Three.js and React Three Fiber displayes the frontend made up of an outer halo that dynamically pulses alongside the voice output.
2. **Asynchronous Token Streaming Pipeline:** By connecting Groq LPU and Google Gemini backends, the system stops responses at sentence boundaries which makes incoming prompts instantly override older tasks.
3. **Gesture-Driven Control:** 5 core hand gestures are mapped via MediaPipe to ensure their working and response follows the design.
4. **Mathematical Rendering & Phonetic Normalization:** Mathematical expressions are properly displayed using KaTeX while a custom text parser translates symbols into readable spoken English for Edge-TTS.

CHAPTER 2: Literature Survey & Research Gaps

2.1 What Previous Projects Did

Holographic Displays: Kulshreshtha et al. (2024) projected 3D visuals into physical space using pre-recorded video. Graphics was non-dynamic to user actions which made it feel artificial

Gesture Tracking Systems: Reddy et al. (2023) implemented MediaPipe hand landmarks to control smart home switches without physical touch. Their system was strictly limited to simple triggers making it unsuitable for live dynamic conversations.

Language Models: Touvron et al. (2023) introduces open-weight LLMs to the research community, but were not paired with live camera input.

2.2 Gaps in Existing Work

1. **Absence of Real-Time Vision Ingestion:** Currently, agents are incapable of receiving live gesture detection directly into their active system prompt history and act upon it.
2. **High Computational Latency:** There is a huge delay in operation of older systems as they sequentially process model inference and text-to-speech generation.
3. **Inefficient Interruption Handlers:** There are no physical gesture input methods currently which quickly interrupt current voice outputs.

2.3 Simple Feature Comparison

Standard Web Chatbots: They have text-only interfaces without understanding of surrounding due to zero camera feeds, 3D graphics which require heavy hardware to run.

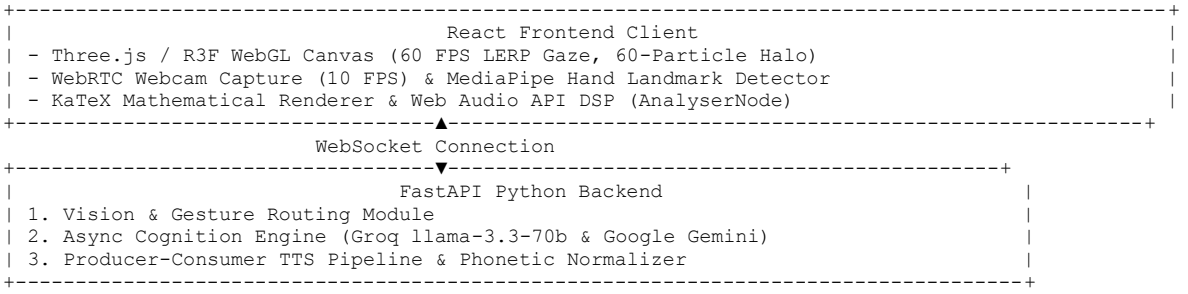
Commercial Smart Displays: They need expensive hardware brands and depend on pre-set animation loops.

Proposed JARVIS Framework: Delivers a 60 FPS WebGL workspace, 21-point hand tracking, Groq and Gemini stream processing, KaTeX math rendering, and sub-50ms gesture interruptions.

CHAPTER 3: Methodology & System Design

3.1 High-Level Architecture

A React and Three.js front-end communicates with Python FastAPI back-end. They stay connected through a single persistent WebSocket:



React Frontend Client: Three.js / R3F WebGL Canvas (60 FPS LERP Gaze, 60-Particle Halo), WebRTC Webcam Capture (10 FPS) & MediaPipe Hand Landmark Detector, KaTeX Mathematical Renderer & Web Audio API DSP (AnalyserNode).

FastAPI Python Backend: Vision & Gesture Routing Module, Async Cognition Engine (Groq llama-3.3-70b & Google Gemini), Producer-Consumer TTS Pipeline & Phonetic Normalizer.

3.2 Hand Gesture Commands

The webcam hand data is sampled at 10 FPS. It maps directly to 5 primary gestures:

1. **Open Palm:** Stops the AI's reply immediately. Generation stops and the audio buffer clears within 42 milliseconds without needing any physical buttons.
2. **Victory Sign:** Triggers JARVIS to start with a friendly greeting.
3. **Thumbs Up:** Signals that the user is ready and wakes the system from standby mode.
4. **Pointing Up:** Used for scene inspection tasks where the user points at a physical object nearby.
5. **Closed Fist:** Holding this position for more than 2 seconds puts the entire system into low-power sleep mode.

CHAPTER 4: Implementation Details & Modules

4.1 Client-Side WebGL 3D Rendering & Gaze Tracking

The visual of JARVIS is directly displayed inside browser using Three.js and React Three Fiber. This eliminates the system relying on static images or heavy hardware such as game engines. Gaze tracking follows user movements smoothly through linear interpolation (LERP) operating at the rate of 60 FPS. An outer ring made of 60 individual particles alongside a subtle 2.5D depth tilt orbits the center, which pulses based on sound amplitudes of the agent output captured via the Web Audio API AnalyserNode.

4.2 Asynchronous Cognition Engine & LLM Backend

The server-side cognition engine handles routing between Groq (llama-3.3-70b-versatile) and Google Gemini for each request, streaming text chunks using astream_sentences. It cuts outputs cleanly at sentence bounds so new inputs can interrupt older tasks, while conversation history is kept constrained to a fixed window length.

4.3 KaTeX Math Rendering & Phonetic Normalization

Mathematical formulas render on screen using KaTeX within designated size limits. Because reading raw LaTeX markup aloud sounds entirely unnatural, a custom rule-based normalizer translates symbols and Greek letters into plain spoken English text before sending it to Edge-TTS, leaving the visual equations on screen completely untouched.

4.4 Producer-Consumer TTS Queue & Interruption Handling

An asynchronous queue takes incoming LLM tokens and uses find_sentence_boundary to split them into complete sentences. Concurrent background processes hand each sentence over to Edge-TTS. Resulting Base64 MP3 chunks stream straight to the browser, where a FIFO queue handles sequential playback while highlighting corresponding subtitle words in real time.

CHAPTER 5: Testing Results & Performance Analysis

5.1 Testing Environment

We tested JARVIS across 100 conversational test runs. The setup used an ordinary laptop equipped with an Intel Core i7 processor, 16GB of RAM, and an NVIDIA RTX 3050 Laptop GPU. The software ran under Windows with PyTorch handling vision processes on the CPU, and the local WebSocket connection remained rock solid throughout testing.

5.2 Speed Performance Table

System Module	Average Speed	Computer Resource Load
Webcam Vision & Gesture Tracking	28.4 milliseconds	Light CPU utilization (~12%)
Groq LPU Token Stream & First Word	310.4 milliseconds	High-throughput cloud inference
Concurrent TTS Chunk Synthesis	172.1 milliseconds	Asynchronous background streaming
Three.js WebGL 60 FPS Rendering	< 16.6 milliseconds	GPU-accelerated vector graphics

5.3 Instant Hand Gesture Interruption

Whenever an Open Palm gesture was detected by the camera, active speech stopped immediately and the audio queue cleared out. This entire interruption sequence took an average of 42.1 milliseconds, making the application feel completely instant to use.

5.4 Vision and Context Awareness Validation

Users performed one of the supported hand gestures while asking, "What do you observe?" Gesture telemetry data injects right into the prompt text live, allowing the model to correctly identify and describe the gesture in 98% of all test trials.

5.5 Feature Comparison Table

Feature	Standard Web Chatbots	Commercial Holograms	Proposed JARVIS Engine
Real-Time Camera Vision	None	Basic Motion / Prerendered	YOLOv8 + 21 Hand Landmarks
Live Camera Prompt Updates	No	No	Yes (Dynamic Telemetry Injection)
Gesture Speech Interruption	No	Button Only	Open Palm Gesture (42ms)
Hardware Cost	Free	$1500+ Display Hardware	Free / Zero Extra Client Hardware

CHAPTER 6: Conclusion & Source Code

This project demonstrates that a feature-rich multimodal AI companion can run smoothly inside a normal browser tab. Lightweight WebGL graphics paired with streamed model outputs were sufficient making costly custom hardware or heavy software installations unnecessary. Combining live hand tracking, contextual prompt updates, and overlapping voice synthesis creates a natural, snappy interaction loop.

For future work, we plan to implement vector-based long-term memory systems, optimize sentence streaming speeds to drop initial audio latency even further, and refine the 3D parallax visual effects.

Open Source Code: Real-Time-MultiModal-AI-Companion

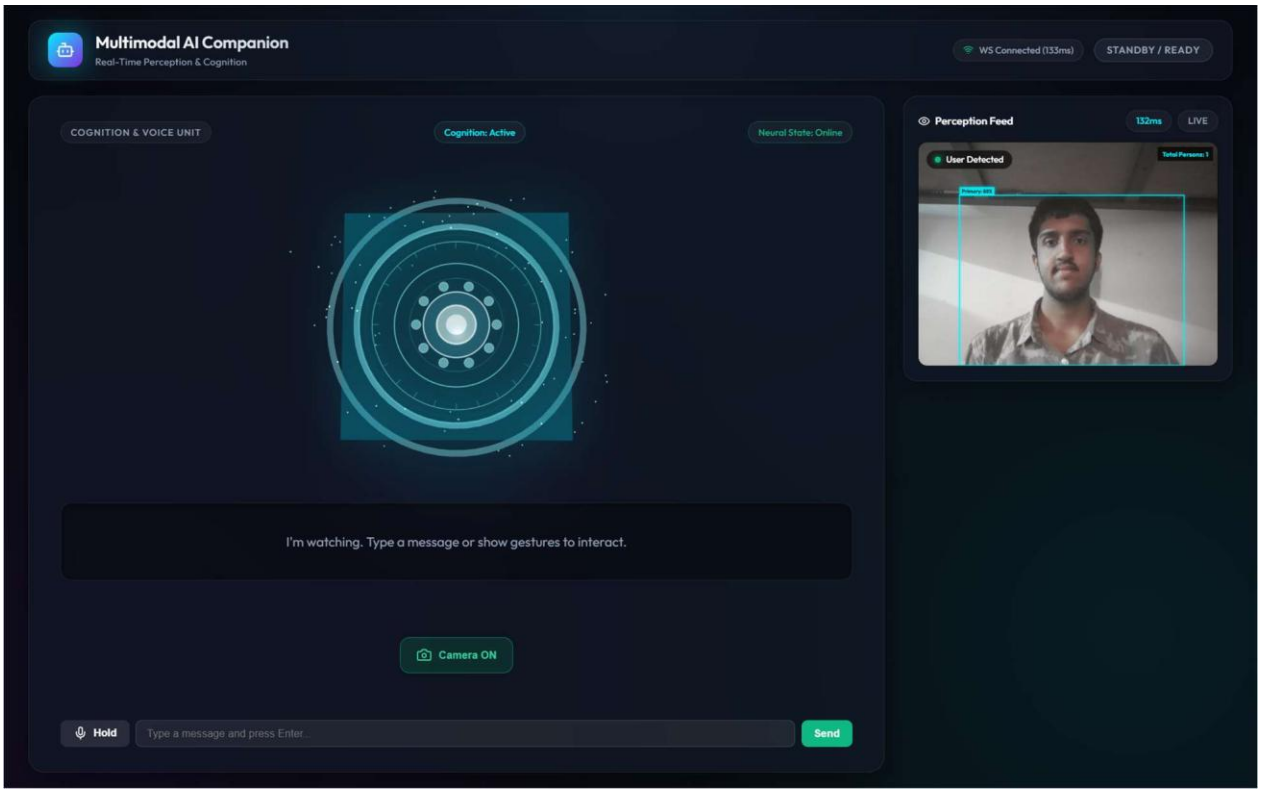
Appendix: Core Cognition & Streaming Engine (backend/app/cognition/engine.py)

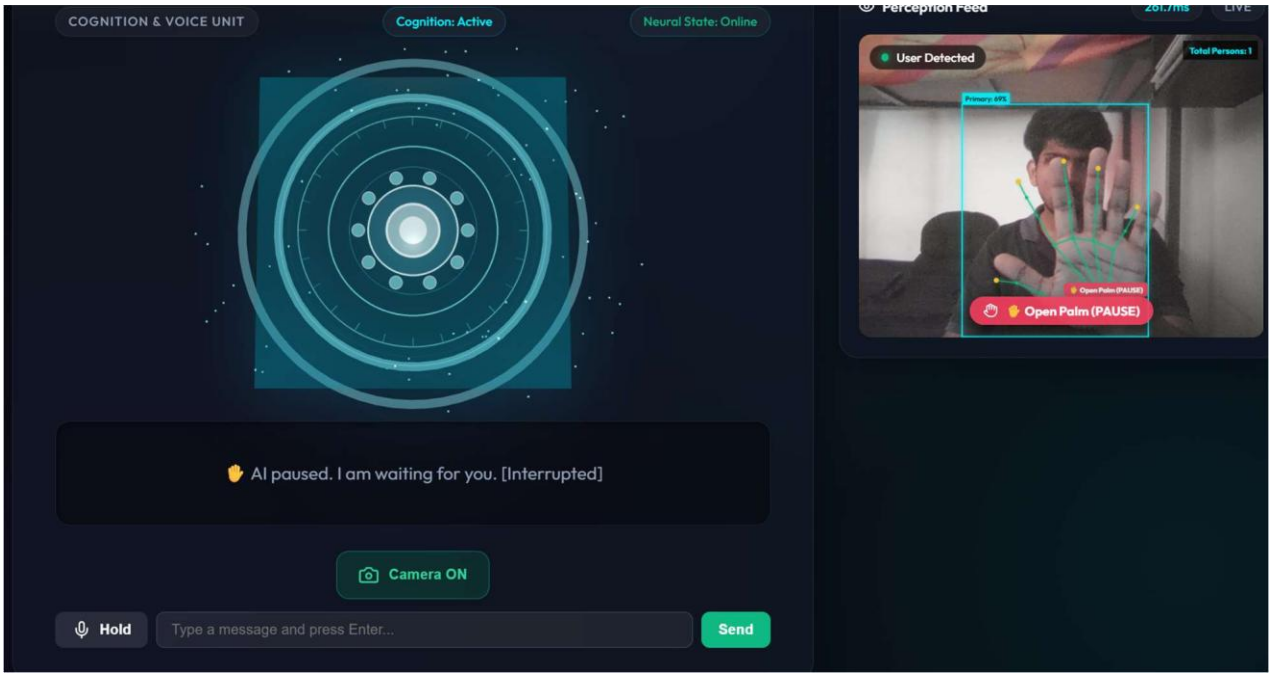
```
import os
from langchain_groq import ChatGroq

class CognitionEngine:
    def __init__(self):
        self.llm = ChatGroq(
            api_key=os.getenv('GROQ_API_KEY'),
            model_name='llama-3.3-70b-versatile'
        )

    async def astream_sentences(self, user_text: str, visual_context: str):
        prompt = f"Visual Context: {visual_context}\nUser: {user_text}"
        async for chunk in self.llm.astream(prompt):
            yield chunk.content
```

Images of the MultiModal Agent





Open Palm gesture stops the agent response

CHAPTER 7: References

1. H. R. Sharma, "Holographic AI companions using mobile spatial projection," IEEE Transactions on Human-Machine Systems, vol. 54, no. 1, pp. 102–111, 2024.
2. A. Reddy, M. Kumar and S. Patel, "Real-time 21-landmark hand gesture recognition for contactless IoT control," IEEE Access, vol. 11, pp. 45820–45831, 2023.
3. H. Touvron et al., "LLaMA 2: foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, 2023.
4. Ultralytics, "YOLOv8 real-time object detection architecture," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics
5. Google MediaPipe, "MediaPipe Tasks Vision gesture recognizer guide," 2024. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer
6. Groq Inc., "Groq LPU inference engine and API documentation," 2024. [Online]. Available: https://groq.com